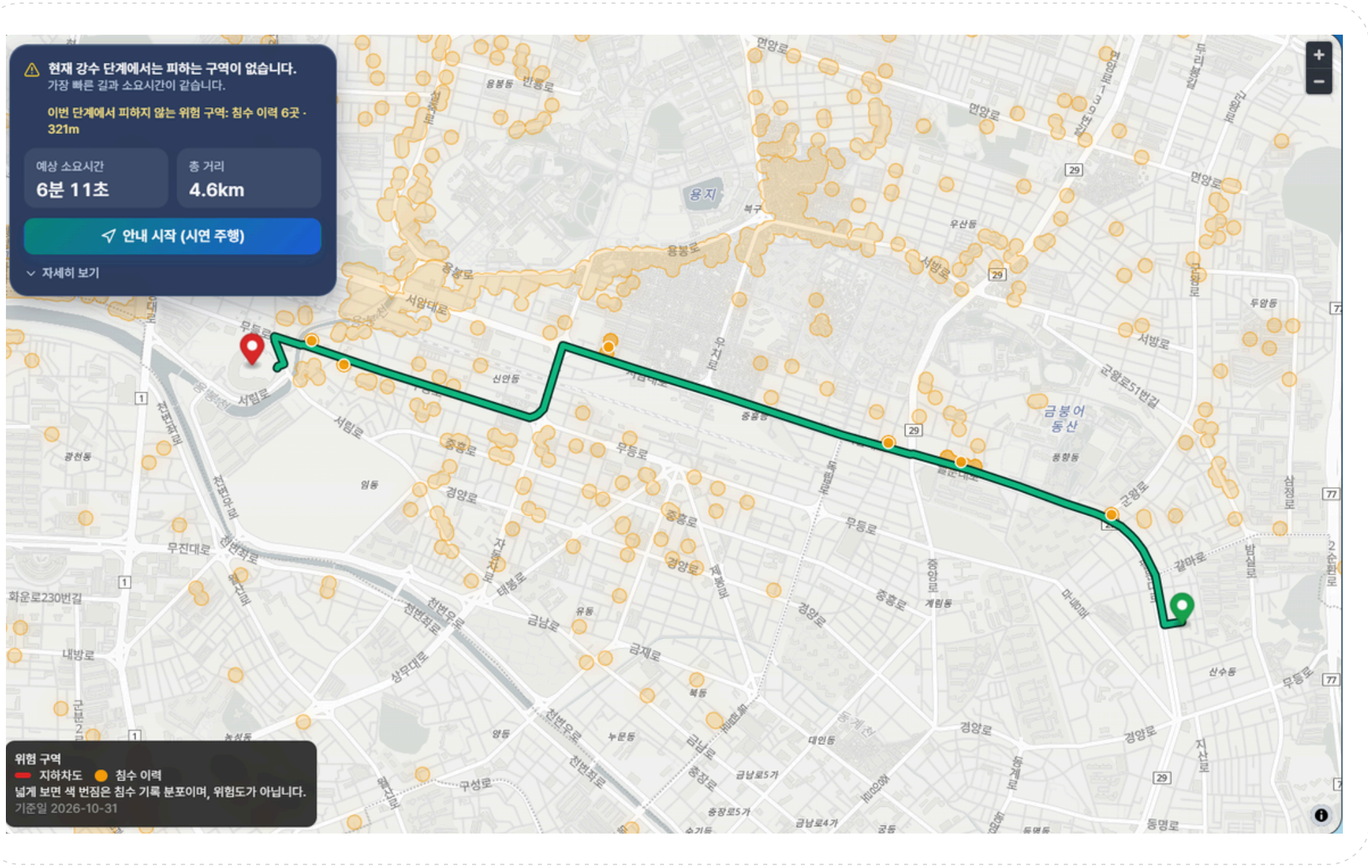
집중호우 회피 내비게이션

호우하우

비가 쏟아지면,
경로가 먼저 움직입니다

호우특보와 실시간 강수량으로 침수 위험 구간을 찾아,
통제되기 전에 피하는 경로를 다시 안내합니다



팀 굳이 이준하 · 김송헌 · 노정균 · 송준호 · 김근환

목차

01 문제 정의

02 문제 해결, 기존 방식과 차별점

03 작동 원리

03 AI와 안전장치

05 Kiro 활용

06 기대효과와 확장

요즘, 비가 확 쏟아진다고 느끼지 않나요?

짧은 시간에 비가 확 쏟아지는 날, 한 번쯤 겪어 보셨을 겁니다

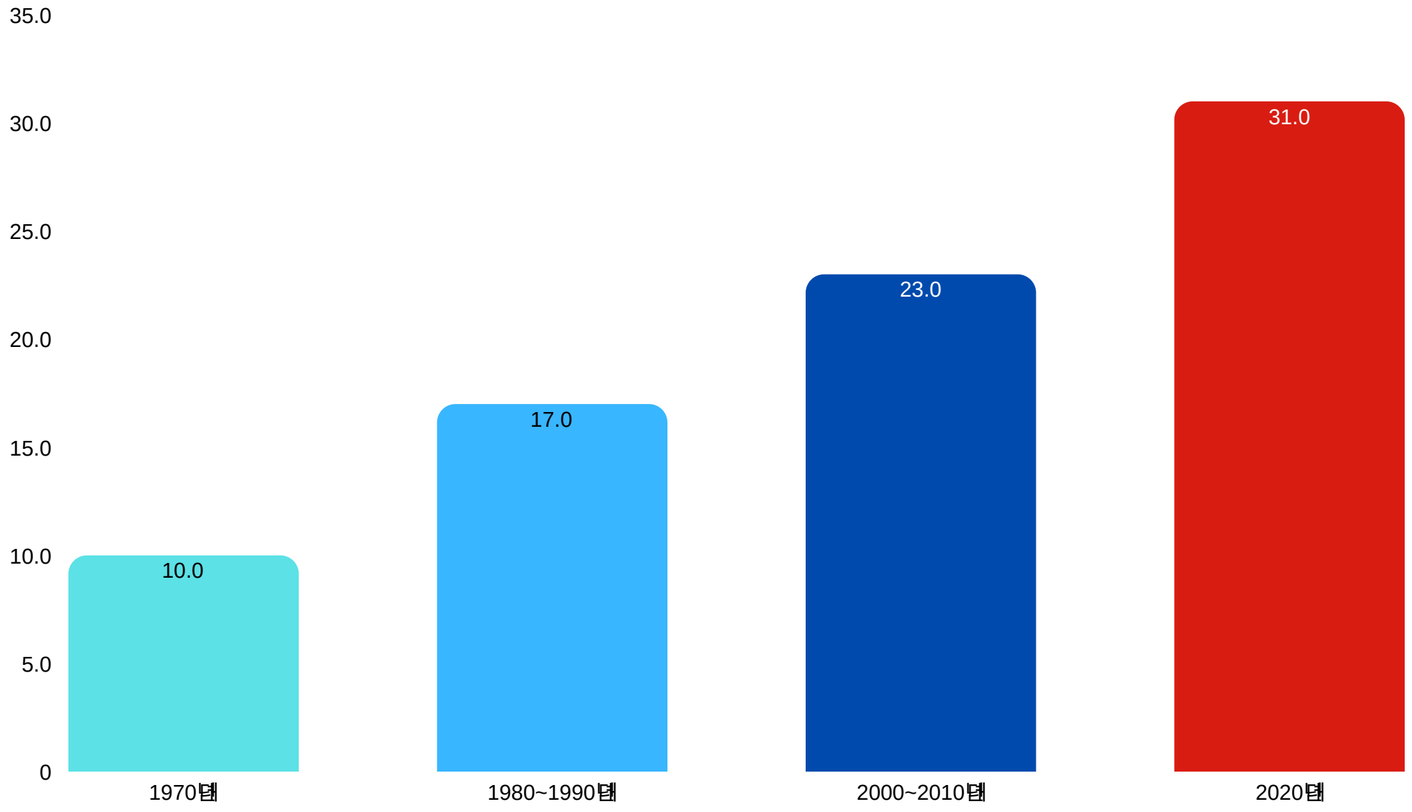
이건 느낌이 아니라,

실제로 바뀌었습니다.

시간당 50mm 이상 집중호우, 1970년대보다 약 3배

연대별 발생 횟수 · 1970년대 10회 → 2020년대 31회

출처: 한국도로공사 자료, 뉴스핌(2026.05.27) 보도



2020년대

31회

1970년대 대비 약 3배

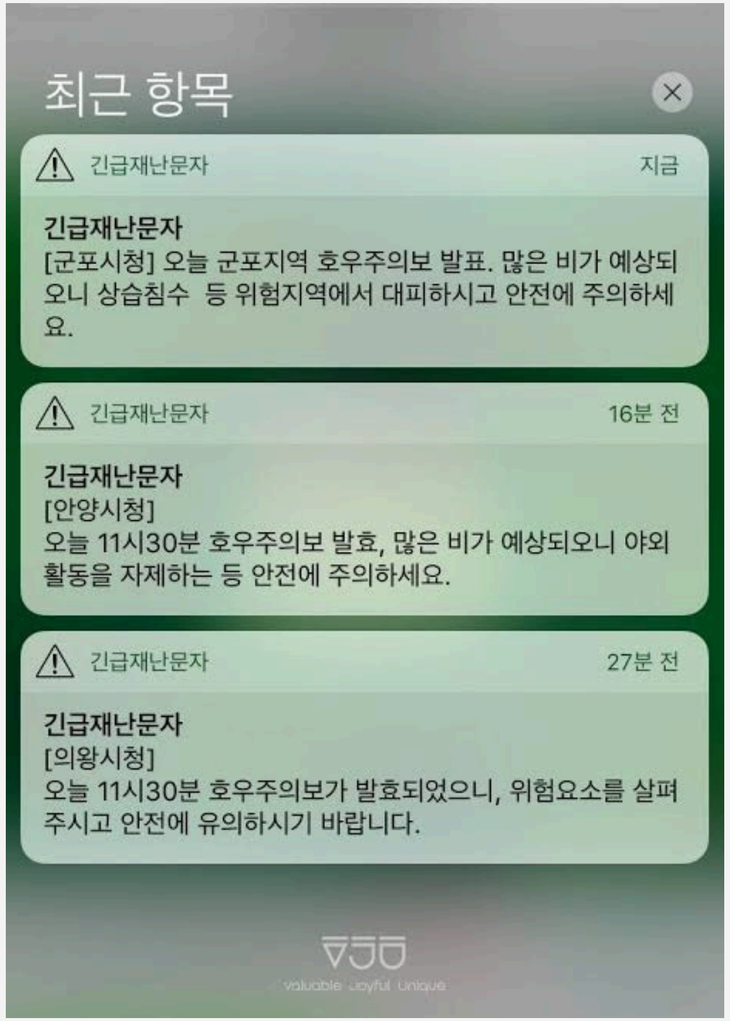
시간당 50mm 이상 집중호우 발생 횟수 (1970년대 10회)

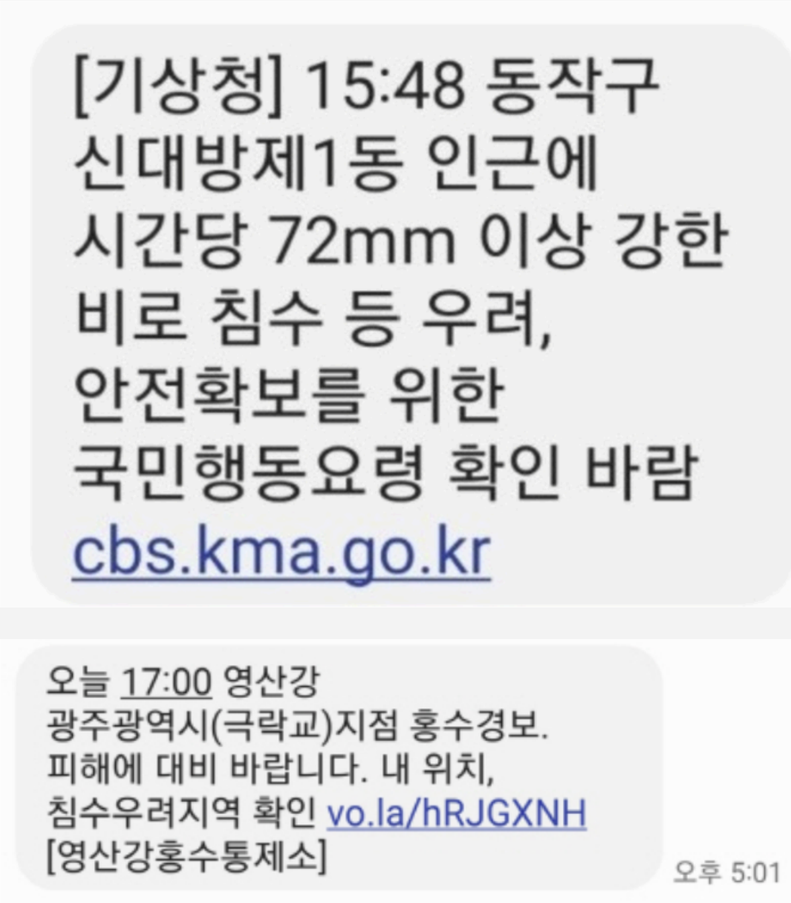
집중 호우 피해, 이제는 먼 이야기가 아닙니다



이 문자를 보고 어떤 생각이 드십니까?

문자는 오지만, 어디를 어떻게 피해야 하는지는 알려 주지 않습니다





운전자의 질문

- '동작구 신대방제1동 인근'은 지금 내 경로 위일까?
- '침수 등 우려' 지역은 정확히 어디일까?
- 운전 중에 링크를 눌러 확인할 수 있을까?

재난문자는 지역 이름을 알려 줄 뿐,
피해야 할 도로는 알려 주지 않습니다

재난에는 시간차가 없습니다

신고와 모니터링을 거쳐 경찰이 통제하기까지 시간차가 생깁니다



침수 발생부터 통제까지 시간차 발생

침수 발생 → 신고 → 모니터링 → **경찰 통제**

통제가 이루어지기 전,
저지대에는 물이 순식간에 모입니다

기존 내비는 이러한 부분들을 반영한
경로 탐색을 하지 않습니다

일반 내비게이션 안내 화면과 집중호우로 침수된 도로 현장

내비게이션 안내 화면

위험 알림이 표시된 경로



집중호우로 잠긴 도로



집중호우 현장

폭우로 도로에 물이 차오른 모습

침수된 실제 주행 도로

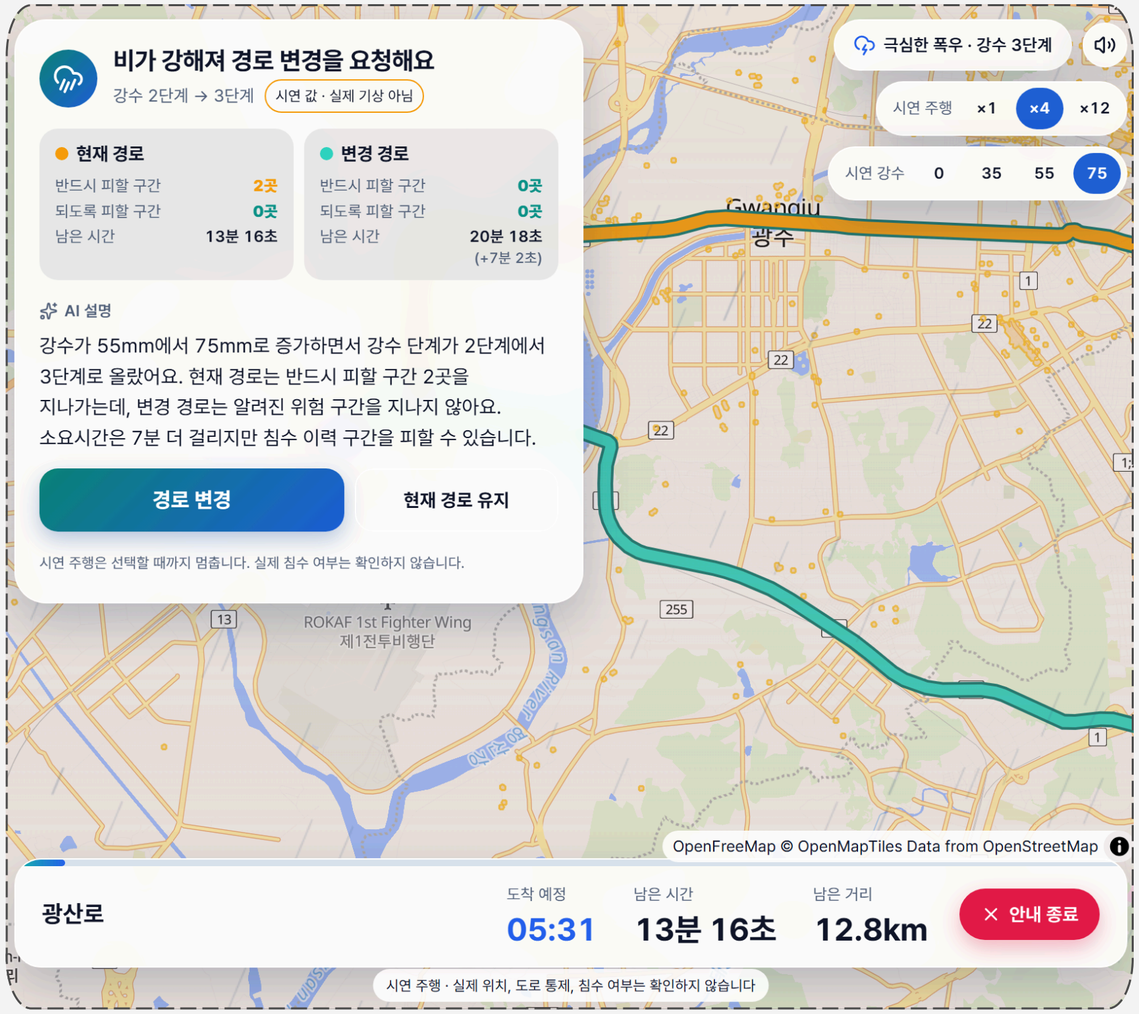


도심 도로 침수

차량이 침수된 도로에 놓인 모습

그래서 통제가 오기 전에, 경로가 먼저 움직입니다

저지대, 침수지역 공공데이터와 호우특보로 위험 구간을 미리 피하는 경로를 다시 찾습니다



침수 발생부터 통제까지 시간차 발생

호우특보 발령 → 저지대 구간 매칭 → 회피 경로 재탐색

✓ 저지대·침수 공공데이터로 위험 구간 사전 파악

✓ 호우주의보가 내리면 즉시 회피 경로 재탐색

특보 감지부터 회피 경로까지 자동으로 순환합니다

공공데이터와 기상 특보를 결합해 위험 구간을 찾고 경로를 다시 계산합니다

1 감지

기상청 호우특보와
실시간 강수량 수신

2 매칭

침수 이력 구역, 지하차도 22곳과
현재 경로 대조

3 계산

단계별 금지 구역 제외 + 소요
시간 + 벌점 최소 선택

4 재탐색·안내

회피 경로 제시 +
위험 구간 음성 경고

입력 데이터

기상청 특보·강수	호우특보·실시간 강수량
침수흔적도	과거에 침수된 구간 기록
수치표고(DEM)	주변보다 낮은 땅 찾기
도로 네트워크	경로를 이루는 도로 구간

네 가지 데이터를 2 매칭 · 3 계산 단계에서 함께 씁니다

후보는 엔진이 만들고, 선택은 강수 단계별 규칙이 합니다

Valhalla가 만든 후보 경로 중에서, 비가 셀수록 더 많은 구역을 피하는 경로를 고릅니다

1 강수 단계 결정

시간당 강수량과 호우특보로 0~3단계를 정합니다

2 후보 경로 모으기

Valhalla가 일반 경로와 회피 경로를 함께 계산합니다

3 금지 구역 거르기

반드시 피할 구역을 지나는 후보를 버립니다

4 추천 경로 선택

소요시간 + 벌점이 가장 작은 경로를 고릅니다

✓ **모든 단계가 같은 후보 묶음에서 고릅니다.** 그래서 비가 더 온다고 추천 경로가 빨라지는 일이 없고, 단계별 결과를 서로 비교하고 검증할 수 있습니다.

비가 셀수록 더 많이 피합니다

시간당 강수량으로 0~3단계를 정하고, 단계마다 피할 구역을 늘립니다

단계	시간당 강수량	반드시 피함 (금지)	되도록 피함 (벌점)
0단계	30mm 미만	없음	없음
1단계	30mm 이상	지하차도	침수 우선 회피 구역
2단계	50mm 이상	지하차도, 침수 우선 회피 구역	그 밖의 침수 이력 구역
3단계	70mm 이상	지하차도, 모든 침수 이력	없음

호우특보가 있으면 단계를 올립니다

호우주의보 → 최소 1단계 · 호우경보 → 최소 2단계

결과는 세 가지로 보여줍니다

회피 완료 · 일부 통과 (금지 구역은 피함)

회피 실패 (경고와 함께 가장 빠른 길 안내)

벌점 = 구역마다 (120초 + 통과 거리 1m당 1초) × 배율 · 1단계: 침수 우선 ×1.5 · 2단계: 침수 일반 ×1.0, 높은 편 ×0.5

AI는 경로를 정하지 않고, 이유를 설명합니다

규칙이 정한 결과만 Bedrock에 넘기고, 검사를 통과한 문장만 보여줍니다

1 사실 확정

구간 수와 추가 시간을
코드가 먼저 계산합니다

2 문장 작성

Bedrock이 그 사실만으로
2~3문장을 씁니다

3 출력 검사

보장·지시 표현과 사실에
없는 숫자를 걸러냅니다

4 화면 표시

통과하면 AI 설명,
아니면 계산된 기본 문장

규칙이 넘긴 사실

강수	**55mm → 75mm**
현재 경로	**반드시 피할 구간 2곳**
변경 경로	**반드시 피할 구간 0곳**
추가 시간	**7분**

AI 설명

강수가 55mm에서 75mm로 증가하면서 강수 단계가 2단계에서 3단계로 올랐어요.
현재 경로는 반드시 피할 구간 2곳을 지나가는데,
변경 경로는 알려진 위험 구간을 지나지 않아요.
소요시간은 7분 더 걸리지만 침수 이력 구간을 피할 수 있습니다.

✓ **AI가 실패해도 경로 안내는 멈추지 않습니다.** 경로는 AI를 기다리지 않고, 실패하면 계산된 문장으로 설명합니다.

Kiro의 **두 가지 협업 기능**을 개발 전 과정에 썼습니다

Spec으로 요구사항을 설계와 작업까지 나누고, Steering으로 코드를 만들 때마다 규칙을 지키게 했습니다

Specs 설계 문서

18 요구사항 requirements.md → **118** 설계 절 design.md → **14** 작업 tasks.md

모든 구현은 tasks.md의 작업에서 시작했습니다

Steering 규칙 파일 5개

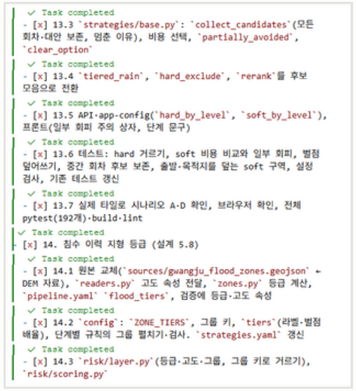
aws.md
AWS 사용 기준 · 항상 적용

data-policy.md
데이터 · 개인정보

deployment.md
배포 규칙

structure.md
폴더 구조

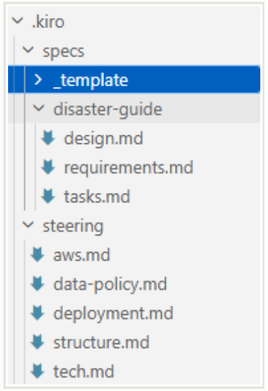
tech.md
기술 스택



tasks.md **작업 완료 기록**

작업마다 완료 표시와 바뀐 파일을 남겼습니다

전체 pytest 192개 · build · lint 확인
작업 13.7 기록



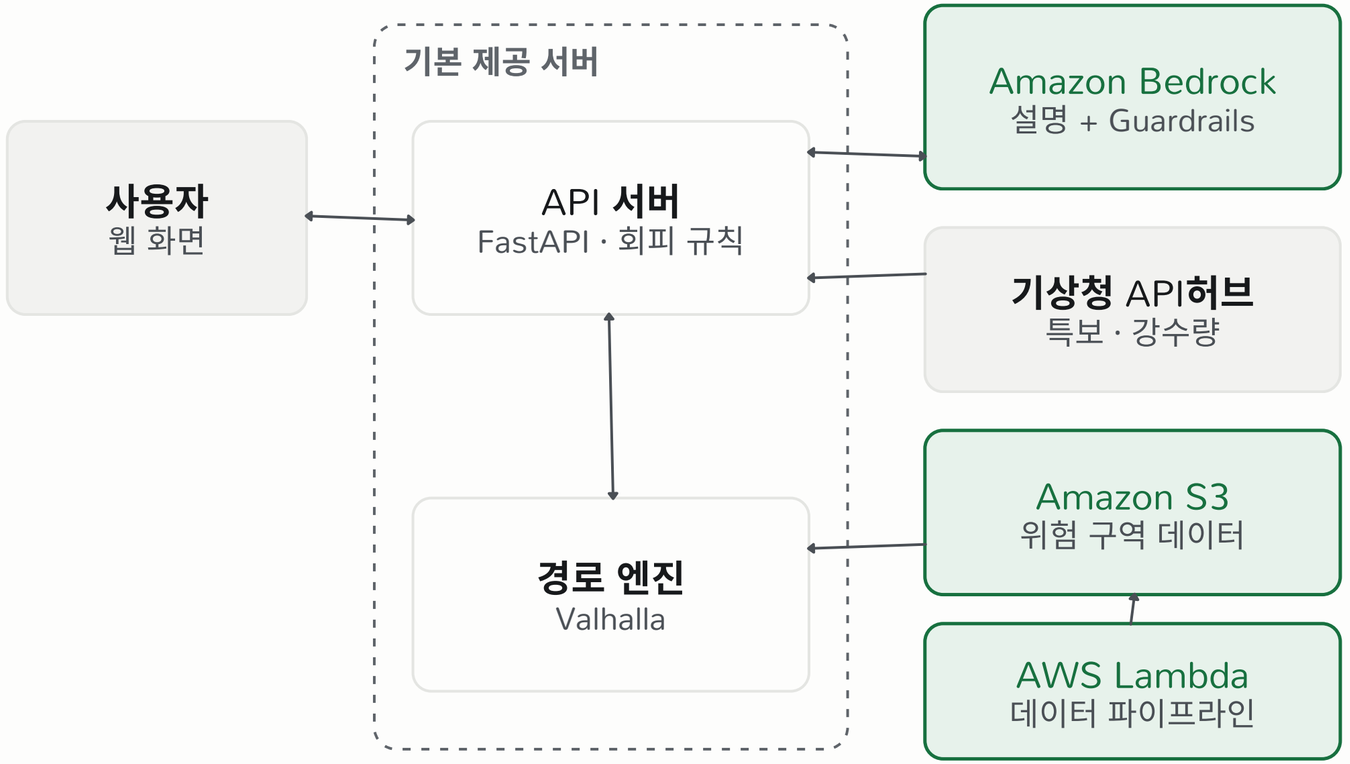
.kiro **폴더 구조**

specs/disaster-guide에 설계 문서 3개
steering에 규칙 파일 5개

Kiro가 코드를 만들 때마다 이 규칙을 함께 읽습니다

계산은 서버에서, 설명은 Bedrock에서

경로는 서버가 규칙으로 정하고, Bedrock은 정해진 결과의 이유만 설명합니다



서비스별 역할과 고른 이유

Amazon Bedrock 회피 이유 설명
Guardrails로 안전 보장 표현 차단

Amazon S3 위험 구역 데이터 보관
파이프라인 결과를 서버와 분리해 관리

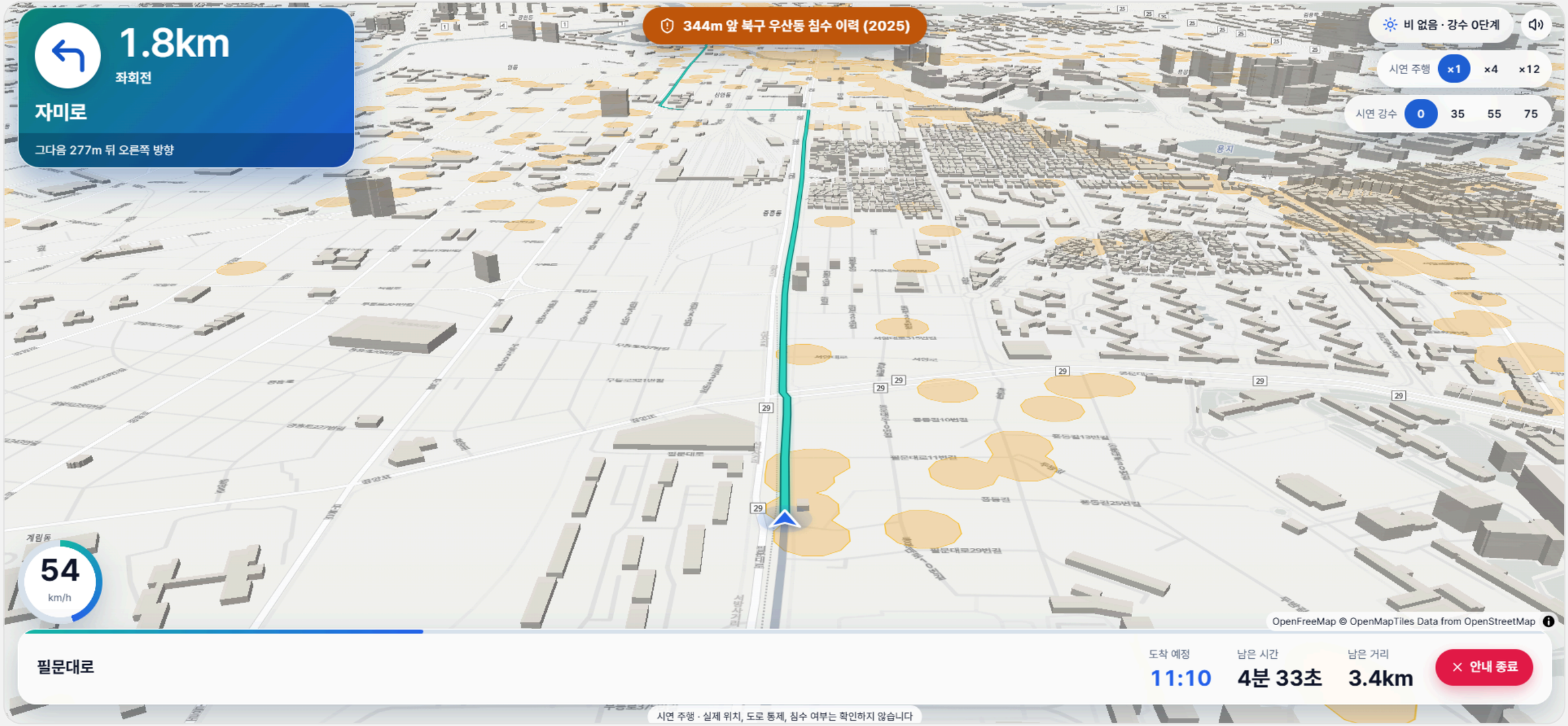
AWS Lambda 데이터 파이프라인
주기 작업을 요청 처리와 분리

기본 제공 서버 FastAPI · Valhalla 실행
경로 엔진은 상시 실행, 메모리 사용 큼

설계 원칙 API 키는 서버에만 · 사용자 위치는 저장하지 않음 · Bedrock이 실패하면 템플릿 문장으로 응답

시연

http://16.208.58.80



운전자부터 사회까지, 함께 달라집니다

경로가 먼저 움직이면, 도로 위의 모두가 덜 위험해집니다

운전자

판단 대신 안내

재난문자를 해석하지 않고
안내만 따라가면 됩니다

판단 부담 감소

지자체·경찰

통제 전 분산

통제가 시작되기 전에
차량이 미리 다른 길로 흩어집니다

현장 대응 부담 감소

경제

침수차 피해 감소

침수차 10대 중 3~4대는
달리다 잠깁니다. 이 부분을 줄입니다

침수차의 30~40%가 대상

사회

더 정확한 위험 지도

우회 기록이 쌓여 공공 침수
데이터를 실시간으로 보완합니다

데이터 선순환

일반 내비와 차이	안내 시점	위험 구간 대응	시연 결과
일반 내비게이션	통제된 뒤에 반영	운전자가 직접 판단	**반드시 피할 구간 2곳 통과**
호우하우	**강수량이 기준을 넘는 즉시**	**위험 구간을 빼고 자동 재탐색**	**반드시 피할 구간 0곳 · 7분 더**

같은 구조로 시간차는 줄이고, 범위는 넓힙니다

감지 → 매칭 → 계산 → 안내 구조는 그대로 두고, 연결할 곳과 데이터만 바꿉니다

1 시간차를 줄인다 — 단기

운전자 → 지자체 역제보

위험 구간 앞 우회·급감속이 몰리면
관제에 '확인 필요' 신호를 보냅니다

정부 재난데이터 플랫폼과 양방향 연동

지하차도 차단 정보는 받아 쓰고
저지대 위험 구간은 돌려줍니다

신고를 기다리지 않는 흐름



2 더 많은 사람에게 — 중기

배달·물류·버스 운행사

관제 화면에서 수십~수백 대 경로를 한 번에

커넥티드카·내비 API

기존 내비에 침수 위험 구간 레이어 제공

보행자·킥보드

같은 위험 지도를 도보·자전거 경로에 적용

지하주차장 경고

목적지가 지하주차장이면 지상 주차 안내

3 더 많은 재난으로 — 장기

재난	1 감지	2 위험 지도
침수 현재 구현	**호우특보·강수량**	**침수흔적도·DEM**
결빙	기온·노면 온도	교량·터널 출입구
산사태	누적 강수량	산사태 위험지도
강풍	강풍특보	가로수 많은 도로

감지·위험 지도 데이터만 바꿔 확장

재난을 피할 때 소외되는 사람은 없어야합니다.

팀원 소개

굳이라고 생각이 드는 아이디어를 개선하고 해결하는 팀입니다.

팀 굳이

이준하	프론트엔드 개발, AI 개발
김송헌	팀장, 기획, 백엔드 개발
김근환	자료조사, 데이터 가공
노정균	서버 관리, 백엔드 개발
송준호	자료조사, 데이터 가공, UI

Q & A

경청해주셔서 감사합니다